



[INCS 617] Software Engineering for Business Informatics

CampusCare

Smart Facility Management System

Final-Milestone: Implementation & Documentation

Frontend · Backend · Project Documentation

Due Date: 17 may

Weight: **40% of Project Grade** (5% SRS + 20% Implementation + 5% Documentation + 10% evaluation)

1. Overview

Congratulations on completing Milestone 1 (SRS Document)! In Milestone 2, you will move from requirements and planning into implementation. This milestone focuses on building the core functionality of the CampusCare application — both the mobile frontend and the backend API — as well as producing comprehensive project documentation.

By the end of this milestone, your team should have a working prototype **that demonstrates end-to-end functionality**: a community member can submit an issue via the mobile app, and a facility manager can view, assign, and manage that issue through the backend system.

Component	Weight	Focus
Frontend & Backend Implementation	20%	Working application
Project Documentation	5%	Complete project docs
Evaluation	10%	
SRS	5%	
Total	40%	

Start with the backend + DB part → then go to the frontend part to make it easier for you to implement the project

Simple example of ReactNative + backend + supabase integration:

<https://github.com/moemen03/Lab-ReactNative>

2. Frontend Requirements (React Native with Expo)

The mobile application must be built using React Native with Expo. All screens should be fully functional and connected to the backend API. The frontend should provide a clean, intuitive user experience for all user roles defined in Milestone 1.

2.1 Authentication Screens

The application must include the following authentication-related screens:

- **Registration Screen:** Allows new users (Community Members, Facility Managers, Workers) to create an account by providing their name, email, password, and selecting their role.

- **Login Screen:** Allows registered users to log in using email and password. Upon successful login, users are navigated to their role-specific dashboard.
- **Logout Functionality:** A logout option accessible from all role dashboards that securely ends the user session and returns to the login screen.
- **Forgot Password Screen (Optional):** Allows users to reset their password by entering their registered email address. The system sends a password reset link or code to the user's email, enabling them to set a new password.

2.2 Community Member Screens

- **Issue Submission Screen:** Allows the community member to submit a new issue by capturing/uploading a photo, entering a description, selecting a category (e.g., Electrical, Plumbing, Cleaning, Furniture), and specifying the location (building, floor, room).
- **My Issues Screen:** Displays a list of all issues submitted by the logged-in community member, showing the issue title/category, current status (Pending, In Progress, Resolved), and submission date.
- **Issue Detail Screen:** Shows the full details of a selected issue, including photo, description, location, category, status, and any updates or worker comments.

2.3 Facility Manager Screens

- **Dashboard / All Issues Screen:** Displays all submitted issues across the system with filtering options (by status, category, date). Each issue shows its current status, assigned worker (if any), and submission date.
- **Issue Assignment Screen:** Allows the facility manager to view an issue's details and assign it to a specific worker from a list of available workers.
- **Issue Management:** The facility manager can update the status of any issue and close resolved issues.
- **Worker Account Management (Optional):** Allows the facility manager to view a list of worker accounts and activate or deactivate them as needed to manage the workforce.

2.4 Worker Screens

- **Assigned Issues Screen:** Displays all issues assigned to the logged-in worker, with their current status.
- **Issue Work Screen:** Allows the worker to view the full issue details, mark the issue as "In Progress," add comments about the work done, and upload a completion photo after fixing the issue.

2.5 System Admin Screens (Optional)

- **User Management Screen:** Displays all registered users with the ability to activate or deactivate accounts.

2.6 Frontend Technical Requirements

- Proper navigation structure using React Navigation (stack and/or tab navigators).
- Role-based navigation: users see only the screens relevant to their role after login.
- Form validation on all input screens (e.g., required fields, email format, password strength).
- Proper error handling and user feedback (loading states, success/error messages).
- Clean and consistent UI/UX design across all screens.
- Image capture/upload functionality for issue submission and completion photos.

3. Backend Requirements (Node.js with RESTful APIs)

The backend must be built using Node.js and expose RESTful APIs that the mobile application will consume. The database must use PostgreSQL (supabase) as specified in the project's technical stack.

3.1 Authentication & Authorization APIs

- **POST /api/auth/register:** Register a new user with name, email, password, and role.
- **POST /api/auth/login:** Authenticate a user and return a JWT token.
- **POST /api/auth/logout:** Invalidate the user's session/token.
- **POST /api/auth/forgot-password (Optional):** Accept a user's email and send a password reset link or code. Include a corresponding reset endpoint to update the password.
- Role-based middleware to protect routes (Community Member, Facility Manager, Worker, Admin).

3.2 Issue Management APIs

Method	Endpoint	Description
POST	/api/issues	Submit a new issue (Community Member)
GET	/api/issues	Get all issues (Facility Manager)
GET	/api/issues/my	Get issues submitted by the logged-in Community Member
GET	/api/issues/:id	Get details of a specific issue
PUT	/api/issues/:id/status	Update the status of an issue (FM / Worker)
PUT	/api/issues/:id/assign	Assign an issue to a worker (Facility Manager)
PUT	/api/issues/:id/close	Close a resolved issue (Facility Manager)
POST	/api/issues/:id/comments	Add a comment to an issue (Worker)
POST	/api/issues/:id/photo	Upload a completion photo (Worker)
DELETE	/api/issues/:id	Delete an issue (Facility Manager)

3.3 Worker Account Management APIs (Optional — Facility Manager)

Method	Endpoint	Description
GET	/api/manager/workers	List all worker accounts (Facility Manager)
PUT	/api/manager/workers/:id/status	Activate or deactivate a worker account (Facility Manager)

3.4 User Management APIs (Optional — Admin)

Method	Endpoint	Description
GET	/api/admin/users	List all registered users
PUT	/api/admin/users/:id/status	Activate or deactivate a user account

3.5 Backend Technical Requirements

The backend implementation must meet the following standards:

1. **Database:** PostgreSQL with proper schema design (users, issues, comments tables with appropriate relationships and constraints).
2. **Authentication:** JWT-based authentication with password hashing (e.g., bcrypt).
3. **Image Storage:** Cloud-based storage service for issue photos and completion photos (e.g., Cloudinary, AWS S3, Firebase Storage, or Supabase Storage).
4. **Input Validation:** Server-side validation for all API endpoints.
5. **Error Handling:** Consistent error response format with appropriate HTTP status codes.
6. **Project Structure:** Clean, organized code with separation of concerns (routes, controllers, models, middleware).

4. Project Documentation (5%)

Your team must produce comprehensive documentation covering the entire project. This documentation should serve as a complete reference for anyone looking to understand, set up, or contribute to the CampusCare application.

4.1 Updated SRS Document

Update your Milestone 1 SRS document to reflect any changes made during implementation. This includes:

- Adding the UML Use-Case Diagram (Section 2.2 from the SRS template).
- Updating any functional requirements that changed during development.
- Ensuring all user stories accurately reflect the implemented features.

4.2 API Documentation

Provide clear and complete documentation for all backend API endpoints. For each endpoint, include:

- HTTP method and URL.
- Request headers, body parameters (with data types), and example request.
- Response format (success and error) with example responses.
- Authentication requirements (which roles can access the endpoint).

4.3 Database Schema Documentation

Include a documented database schema with:

- Entity-Relationship Diagram (ERD) showing all tables and their relationships.
- Table definitions: column names, data types, constraints, and foreign keys.
- Any seed/sample data used for testing.

4.4 Setup & Installation Guide

A README-style guide that explains how to set up and run the project locally. This must include:

- Prerequisites (Node.js version, Expo CLI, PostgreSQL or supabase, etc.).
- Step-by-step instructions to install dependencies, configure environment variables, set up the database, and run both the frontend and backend.
- Any third-party service setup required (e.g., cloud storage credentials).

4.5 Project Structure Overview

A brief explanation of the project's folder structure for both the frontend and backend, describing the purpose of key directories and files.

5. Deliverables Summary

Each team must submit the following by the deadline:

#	Deliverable	Details
1	Source Code	Complete frontend (React Native/Expo) and backend (Node.js) source code, pushed to a shared Git repository (e.g., GitHub).
2	Working Demo	The application must be runnable and demonstrable.
3	Updated SRS	Revised SRS document reflecting any implementation changes, including the UML Use-Case Diagram.
4	API Documentation	Complete documentation of all API endpoints with request/response examples.
5	Database Schema	ERD and table definitions document.
6	Setup Guide	README with installation and running instructions.

6. User Story Coverage

The following table maps the user stories from Milestone 1 to the expected implementation in Milestone 2. All High-priority items from the Product Backlog must be fully implemented. Medium-priority items are expected. Low-priority (System Admin) items are optional but encouraged.

Story ID	Description	Priority
US-CM-01	Community Member account creation	High
US-CM-02	Community Member login	High
US-CM-03	Submit issue with details	High
US-CM-04	Add description to issue	High
US-CM-05	View submitted issues	High
US-CM-06	Community Member logout	Medium
US-CM-07	View issue status	High
US-FM-01	Facility Manager account creation	High
US-FM-02	Facility Manager login	High
US-FM-03	Facility Manager logout	Medium
US-FM-04	View all submitted issues	High

Story ID	Description	Priority
US-FM-05	Update issue status	High
US-FM-06	Assign issue to worker	High
US-FM-07	Close resolved issue	High
US-W-01	Worker account creation	High
US-W-02	Worker login	High
US-W-03	Worker logout	Medium
US-W-04	View assigned issues	High
US-W-05	Comment on issue	Medium
US-W-06	Mark issue as In Progress	High
US-W-07	Upload completion photo	Medium
US-SA-*	System Admin features (all)	Low (Optional)

7. Important Notes

Version Control: All team members must contribute to the Git repository. Commit history will be reviewed to assess individual contributions.

Code Quality: Write clean, readable code with consistent naming conventions and adequate comments where necessary.

Live Demo: Teams will be required to present a live demo of their application during the tutorial session. Ensure the app runs without critical errors.

Collaboration: This is a team effort. Maintain clear communication, divide work fairly, and use project management tools (e.g., Trello, Jira, or GitHub Projects) to track tasks.

Cheating Cases: Any form of cheating — including but not limited to copying code from other teams, submitting work done by someone outside the team, purchasing or outsourcing project deliverables, or presenting another team's work as your own — will be treated as a serious academic offense. All involved parties will receive a zero on the entire project and the case will be escalated to the university's academic integrity committee for further disciplinary action.

Good luck! *This is where your project comes to life. Build something you're proud of.*